

CPDS Laboratory Assignment

Laboratory Assignment 0: Parallel Execution Environment

J.R. Herrero

2026



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Acknowledgment of past contributions: E. Ayguadé and D. Jiménez

Contents

1	Experimental setup	2
1.1	Node architecture and memory	3
1.2	Execution modes: interactive vs queued	4
1.3	Serial compilation and execution	5
1.4	Compilation and execution of OpenMP programs	7
1.4.1	Compiling OpenMP programs	7
1.4.2	Executing OpenMP programs	7
1.4.3	Strong vs. weak scalability	8
1.5	Compiling and Executing MPI programs	8
1.5.1	Strong scalability with processes	8
1.5.2	Strong scalability with processes and threads	9

Deliverable

Session 1

Experimental setup

The objective of this first assignment is to familiarise yourself with the hardware and software environment that you will be using during the semester to do all laboratory assignments in CPDS. From your PC/terminal booted with Linux¹ you will access **boada**, a multiprocessor server located at the Computer Architecture Department at UPC. From a machine within the UPC network you will be able to access **boada** directly. However, to be able to access the **boada** server from a computer outside the UPC you will need to use a UPC VPN connection. To connect to it you will have to establish a connection using the secure shell command: "`ssh -X cpds1XYZ@boada.ac.upc.edu`", being **cpds1XYZ** the username assigned to you. Option `-X` is necessary in order to forward the X11 data and be able to open remote windows in your local desktop². You can change the password for your account using the following command: "`ssh -t cpds1XYZ@boada.ac.upc.edu passwd`"³.

Once you are logged in you will find yourself in one of the interactive nodes: **boada-11** to **boada-13**, where you can execute interactive jobs and from where you can submit execution jobs to the rest of the nodes in the machine. In fact, **boada** is composed of several nodes (named **boada-1** to **boada-20**), equipped with five different processor generations. The following table gives a partial view of the cluster:

Node name	Processor generation	Interactive	Partition
boada-[6-8]	Intel Xeon E5-2609 v4	No	execution2
boada-9	Intel Xeon E5-1620 v4 + Nvidia K40c	No	cuda9
boada-10	Intel Xeon Silver 4314 + 4 x NVIDIA GeForce RTX 3080 + 1 x NVIDIA GeForce RTX 4090 + 3 x NVIDIA RTX PRO™ 4000 Blackwell	No	cuda
boada-[11-13]	Intel Xeon Silver 4210R	Yes	interactive
boada-15	Intel Xeon Silver 4210R + ASUS AI CRL-G116U-P3DF	No	iacard
boada-[18-20]	Intel Xeon Silver 4314	No	execution

However, in this course you are going to use only nodes **boada-11** to **boada-13** interactively, nodes **boada-18** to **boada-20** through the **execution** queue and, later on, node **boada-10** through the **cuda** queue. Submission of jobs for *batched* execution in a queue is explained in the next section. The rest of the nodes have restricted access and CPDS users are not allowed to send jobs to their corresponding queues (Partitions). Note that all the nodes within a particular **Partition** are identical. Therefore, you do not need to worry if the execution was done in node **boada-18**, **boada-19**, or **boada-20**, when submitting to the **execution** queue.

All nodes have access to a shared NAS (*Network-Attached Storage*) disk; you can access it through

¹You can also access from your laptop, booted with Linux, Windows or MacOS X, if a secure shell client is installed.

²Option `-Y` if you are connecting from a MacOS X laptop with XQuartz. With Windows, if you use putty (<https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>) then you will also need xming: (<https://wiki.centos.org/HowTos/Xming>). Alternatively, you can use MobaXterm (<https://mobaxterm.mobatek.net/download.html>) which has everything integrated.

³The `passwd` command will be executed in **boada**. After entering the old password correctly twice you will be asked for your new password also twice.

/scratch/nas/1/cpdsXXYY (in fact this is your *home directory*, check by typing `pwd` in the command line). In addition, each node in **boada** has its own local disk which can be used to store temporary files non visible to other nodes; you can access it through /scratch/1/cpdsXXYY.

We will post all necessary files to do each laboratory assignment in /scratch/nas/1/cpds0/sessions. For the session today, you need to extract the files from file **a0.tar.gz** from that location by uncompress it **into** your home directory in **boada**.

From the the root of your home directory unpack the files with the following command line:
`"tar -zxvf /scratch/nas/1/cpds0/sessions/a0.tar.gz".`

In order to set up all environment variables you have to process the `environment.bash` file now available in your home directory with `"source ~/environment.bash"`. **Note:** since you have to do this every time you login in the account or open a new console window, we recommend that you add this command line in the `.bashrc` file in your home directory, a file that is executed every time a new session is initiated. If you have unpacked the `a0.tar.gz` files in your home directory then this file has already been added automatically to your home directory and it will be used from your next connection to **boada**. From the next time you log in to **boada** it will be automatically read, and environment variables will be properly set effortless.

In case you need to transfer files from **boada** to your local machine (laptop or desktop in laboratory room), or viceversa, you have to use the secure copy `scp` command. For example if you type the following command `"scp -O cpds1XYZ@boada.ac.upc.edu:a0/pi/pi_seq.c ."` in your local machine you will be copying the source file `pi_seq.c` located in directory `a0/pi` from your home directory in **boada** to the current directory, represented with the `"."`, in the local machine, with the same name. You may also be interested in the `"-r"` flag to recursively copy a subdirectory.

1.1 Node architecture and memory

The first thing you will do is to investigate the architecture of the available nodes in **boada**. Run `sbatch submit-arch.sh` command in directory `a0/arch`. This command will enqueue `submit-arch.sh` script. More detail about queued execution below. This script will execute the `lscpu`, `lstopo` and `numactl` commands in order to obtain information about the hardware in one of the nodes in the execution queue (**boada-18** to **20**). The execution of these three commands using `submit-arch.sh` script generates four files, where **number** may be 18,19 or 20: 1) `lscpu-boada-number`, 2) `lstopo-boada-number`, 3) `map-boada-number.fig` and 4) `numactl-boada-number`. You can use the `xfig` command to visualise the output file generated (`map-boada-number.fig`) and export to a different format (PDF or JPG, for example) using **File** → **Export** in order to include it in your deliverable for this laboratory assignment⁴. Those files will help you to figure out:

- the number of sockets, cores per socket and threads per core in a specific node;
- the amount of main memory in a specific node, and each NUMA node: each node in **boada** internally has 2 NUMA⁵ nodes (one per socket). Note the difference between *node* and *NUMA node*! Our cluster, **boada** has several nodes, which are basically several different machines within one. And each of them, has a mother board with several sockets where we plug chips containing processors; and several slots where to plug memory modules. Even when both chips can access all the memory plugged to the same mother board, some memory modules are closer and faster to access than the other memory modules. This motivates the name *Non-Uniform Memory Access time (NUMA)*.
- the cache memory hierarchy (L1, L2 and L3), private or shared to each core/socket.

⁴In the **boada** Linux distribution you can use `xpdf` to open pdf files and `display` to visualise graphics files. You can also use the `"fig2dev -L pdf map.fig map.pdf"` command to convert from `.fig` to `.pdf`; look for alternative output graphic languages by typing `"man fig2dev"`.

⁵Non-Uniform Memory Access, or NUMA, is a computer memory design used in *shared memory* multiprocessor systems where the time it takes for a processor to access memory depends on the memory's location. A processor can access its local memory (memory attached to the same processor or node) faster than it can access non-local memory (memory attached to another processor or node). This architecture helps reduce the performance bottleneck that can occur when multiple processors all try to access a single, shared memory resource simultaneously.

- the NUMA topology within a node. The `numactl --hardware` command shows a system's NUMA topology, including a final matrix, based on the ACPI System Locality Information Table (SLIT), with *node distances* that provides a relative measure of memory access latency. The value at (row, column) represents the relative latency for a CPU on the row node to access memory on the column node. The values are unitless and are scaled relative to the latency of a processor accessing its own *local memory*, which is normalized to 10. The matrix is symmetric, so the latency from node 1 to node 0 is the same as from node 0 to node 1. This matrix gives you a quick, reliable way to understand the performance penalty of *remote memory* access, which is the core concept of a NUMA architecture.

Which is the number of threads per core? If *Hyper-threading*⁶ is implemented in the microprocessor and it's enabled at boot time, then the system reports 2 threads per core. Hyper-threading exposes two execution contexts (logical cores) for each physical core, making the operating system see more processors than are physically present.

Fill in the table in the "Deliverable" section with the main characteristics of the node. Draw the architecture of the node based on the information generated by the tools above. The "--of fig map.fig" option for `lstopo` can be very useful for that purpose.

1.2 Execution modes: interactive vs queued

There are two ways to execute your programs in boada:

1. via the Slurm⁷ queueing system (in one of the nodes `boada-18` to `boada-20`);
2. interactively (in any of the login nodes `boada-11` to `boada-13`). In this case, even when the hardware in these nodes consists of several cores, the system is configured to limit the number of cores to be used in parallel executions to one (a single core with 2 hardware threads thanks to *Hyper-threading*).

It is mandatory to use option 1 when you want to execute scripts that require several processors in a node, ensuring that your job is executed in isolation (and therefore reporting reliable performance results) and to avoid adding additional load to the interactive node accessed by all users; the execution starts as soon as a node is available. When using option 2 your execution starts immediately but it will be limited to using only 1 core and it will share resources with other programs and interactive jobs, not ensuring representative timing results. Usually, scripts for both options will be provided (`submit-xxxx.sh` for option 1 and `run-xxxx.sh` for option 2, respectively):

- Queueing a job for execution: `sbatch [-p partition] ./submit-xxxx.sh`. Additional parameters may be specified, if needed by the script, after the script name. If you do not specify the name of the partition with `-p partition` your script will run on the **execution** partition by default. Use `squeue -u your_username` to ask the system about the status of your current jobs. You can use `scancel` followed by the job identifier to remove a job from the queueing system. Note that partition names associated to each node name are shown in the last column of the table above. The execution of your job will take place once a node associated to the specified partition becomes available. After job termination `squeue` will not list it anymore. Then, you can check the contents of your directory. *Hint*: Use command `ls -ltr` to list files chronologically. In addition to the files being generated by the execution of the script `submit-xxxx.sh`, two additional files will be created. Their name will have the script name followed by an `.e` or an `.o` and the job identifier. They will contain the messages sent to the **standard error** and **standard output** respectively during the execution of the job. You should check them to be sure that there were no errors and the results make sense.

⁶ *Hyper-threading* is Intel's technology that allows a single physical CPU core to act as two logical cores. This *simultaneous multithreading (SMT)* technique can improve processor efficiency and throughput: it capitalizes on situations where a core might otherwise be waiting for data or the next instruction, using that "idle time" to process another thread, leading to better multitasking performance and responsiveness for demanding *memory bound* applications and workloads.

⁷ Slurm (Simple Linux Utility for Resource Management) is an open-source workload manager and cluster management software used in high-performance computing (HPC) environments to schedule and allocate compute resources fairly among users.

- Interactive execution: `./run-xxxx.sh`. Additional parameters may be specified after the script name. Jobs interactively executed have a short time limit to be executed.

1.3 Serial compilation and execution

Next you will get familiar with the compilation and execution steps for both sequential and parallel applications. For this, we will use some codes which compute digits of number Pi. As a reference, Figure 1.1 shows a python code which computes the area under one quadrant of the unit circle, i.e. a circle of radius equal to 1, defined by the curve $y = \sqrt{1 - x^2}$.

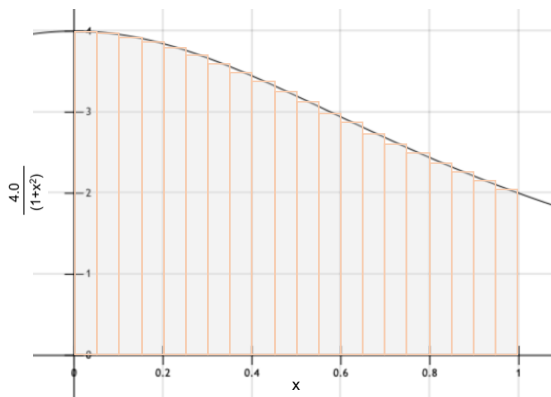
```
def midpoint_riemann_sum():
    N = 1000000000

    delta_x = (1 - 0) / N
    x = 0
    pi = 0
    start = time.time()

    while x < (1-delta_x):
        x_next = x + delta_x
        x_mid = (x_next + x) / 2
        f_x = math.sqrt(1 - math.pow(x_mid, 2))
        pi += f_x * delta_x
        x += delta_x

    pi = 4 * pi
    end = time.time()
    print(end - start)
    print("Estimate: " + str(pi))
    print("Actual:   " + str(math.pi))
```

Figure 1.1: Python code for computing Pi (unit circle)



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^N F(x_i) \Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i .

Figure 1.2: Another way to compute Pi (derivative of the arctangent)

The execution time for this code on `boada-1` took 522.1 seconds to compute the estimate 3.14159266367805 (actual value is 3.141592653589793). As you can see, it took a long time while it only computed 7 correct digits of Pi. We aim at estimating digits of Pi much faster. For that we are going to use some C codes which we will parallelize. You are going to use a very simple code, `pi_seq.c`, which you can find inside the `a0/OpenMP/pi` directory. The code in `pi_seq.c` performs the computation of the Pi number

by computing the integral of another equation, the one shown in Figure 1.2. This time the algorithm is using the derivative of the arctangent function of x , which is equal to $1/(1+x^2)$ which is another way to estimate Pi. Again, the equation can be solved by computing the area defined by the function, which at its turn can be approximated by dividing the area into small rectangles and adding up its area. This is one of the many ways to estimate digits of number Pi. The algorithms above multiply by 4 because both functions compute $\pi/4$. Figure 1.3 shows a simplified version of the code you have in `pi_seq.c`. Variable `num_steps` defines the number of rectangles, whose area is computed in each iteration of the `i` loop.

```
static long num_steps = 100000;
void main () {
    double x, pi, step, sum = 0.0;

    step = 1.0/(double) num_steps;
    for (long int i=0; i<num_steps; ++i) {
        x = (i+0.5)*step;
        sum += 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

Figure 1.3: Serial C code for computing Pi (derivative of the arctangent)

Figure 1.4 shows the compilation and execution flow for a sequential program. We will compile programs to generate binary executable files through a **Makefile**, with multiple targets that specify the rules to compile each program version. We are going to use `gcc` (the C front-end from the *GNU Compiler Collection*).

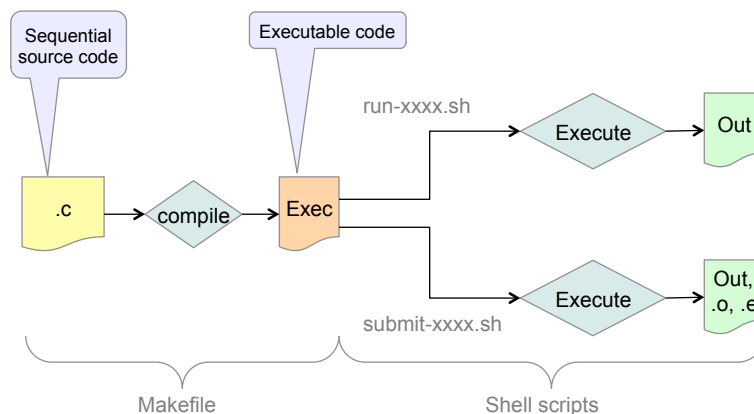


Figure 1.4: Compilation and execution flow for a sequential program.

In the following steps you will compile `pi_seq.c` using the **Makefile** and execute it interactively and through the queueing system, with the appropriate timing commands to measure its execution time:

1. Open the **Makefile** file, identify the **target** you have to use to compile the sequential code. Observe how the compiler is invoked. Execute the command line `"make target_identified"` in order to generate the binary executable file. Alternatively, you can just type `"make"` which will compile all programs.
2. Interactively execute the binary generated to compute number pi sequentially by doing 1.000.000.000 iterations using the `run-seq.sh` script (`"run-seq.sh pi_seq 1000000000"`) which returns the user and system CPU time, the **elapsed** time, also called **wall** time, and the % of CPU used (using the GNU time program in `/usr/bin/time`). In addition, the program itself also reports the elapsed execution time using `gettimeofday`. Look at the source code and identify the function invocations and data structures required to measure execution time. Note that CPU time is the quantity of

processor time taken by the process. This does not indicate duration. Elapsed time represents the total duration of the execution. If we exploit parallelism, elapsed time can be less than the total CPU time because several cores are used to do the work and it can get done faster.

3. Submit the execution to the queueing system using the "`sbatch submit-seq.sh`" command and use "`squeue`" to see that your script is running. Look at `submit-seq.sh` script and the results generated (the standard output and error of the script and the `time-pi_seq-boada{18-20}` file).

1.4 Compilation and execution of OpenMP programs

In this section we are going to use `OpenMP`, the standard for parallel programming using shared-memory, to express parallelism in the C programming language. Note that `OpenMP` will be explained in detail later in this course, and the goal now is to have a first contact and observe that we can speed-up the execution of our programs. Therefore, in this section we will see how to compile and execute parallel programs in `OpenMP` and, next, observe the performance improvements. Figure 1.5 shows the compilation and execution flow for an `OpenMP` program. The main difference with the flow shown in Figure 1.4 is that now the `Makefile` will include the appropriate compilation flag to enable `OpenMP`.

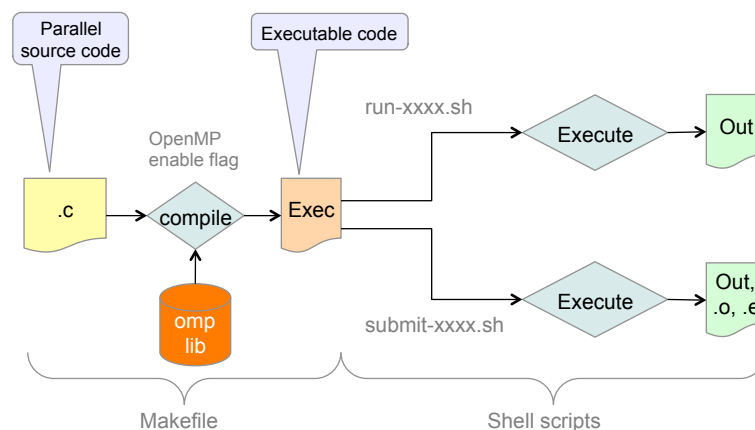


Figure 1.5: Compilation and execution flow for OpenMP.

1.4.1 Compiling OpenMP programs

1. In the same `a0/OpenMP/pi` directory you will find an `OpenMP` version of the code for doing the computation of pi in parallel (`pi_omp.c`). Compile the `OpenMP` code using the appropriate target in the `Makefile`). What is the compiler telling you? Is the compiler issuing a warning or an error message? Is the compiler generating an executable file? The compilation run smoothly because we used flag `-fopenmp`, but if you remove the `-fopenmp` flag and force the recompilation you will see the difference.

1.4.2 Executing OpenMP programs

1. Interactively execute the `OpenMP` code with 8 threads (processors) and the same number of iterations (1.000.000.000) using the `run-omp.sh` script ("`run-omp.sh pi_omp 1000000000 8`"). What is the `time` command telling you about the user and system CPU time, the elapsed time, and the % of CPU used? Take a look at the script to discover how do we specify the number of threads to use in `OpenMP`. Note that, since any login node `boada-[11-13]` is shared by many users and is reserved for interactive usage, it has been configured so that our processes only have 1 core available for their execution. Thus, we will not benefit from the parallelization with many threads on the interactive nodes.

2. Use `submit-omp.sh` script to queue the execution of the OpenMP code ("`sbatch submit-omp.sh`") and measure the CPU time, elapsed time and % of CPU when executing the OpenMP program using 8 threads in isolation. **Do you see a major difference between interactive and queued execution?**

1.4.3 Strong vs. weak scalability

Finally in this section you are going to explore the scalability of the `pi_omp.c` code when varying the number of threads used to execute the parallel code. To evaluate the scalability the ratio between the sequential and the parallel execution times will be computed. Two different scenarios will be considered: *strong* and *weak* scalability.

- In *strong* scalability the number of threads is changed with a fixed problem size. In this case parallelism is used to reduce the execution time of your program.
- In *weak* scalability the problem size is proportional to the number of threads. In this case parallelism is used to increase the problem size for which your program is executed.

We provide you with two scripts, `submit-strong-omp.sh` and `submit-weak-omp.sh`, which should be submitted to the queueing system with the `sbatch` command. The scripts execute the parallel code using from 1 (`np_NMIN`) to 32 (`np_NMAX`) threads. The problem size for strong scalability is 1.000.000.000 iterations; for weak scalability, the initial problem size is 100.000.000 which grows proportionally with the number of threads. The execution **will take some time** because several executions are done for each test (in order to get a minimum time), please be patient! And, very important, do NOT execute interactively but, instead, **submit them for execution in the execution queue**. As a result each script generates several files, including a plot (a file with a name ending in `.ps` in Postscript format) showing the resulting parallel execution time and speed-up or parallel efficiency. **Visualise⁸ the plots generated and reason about how the speed-up changes with the number of threads in the two scenarios.**

Edit `submit-strong-omp.sh`, change the value of `np_NMAX` from 32 to 64 and execute again. **Can you explain the behaviour observed in the scalability plot? Even when Hyper-Threading is enabled and the system can take instructions quickly from 64 threads, there are only 32 cores available! Hint: Are we currently benchmarking a *memory bound* application or a *CPU bound* application?**

1.5 Compiling and Executing MPI programs

Next, we will test how to compile and execute parallel programs in MPI, or a combination of both MPI and OpenMP. To do so, follow similar steps as for OpenMP in the previous section, but working with the files in directory `a0/MPI/pi`. The code calculates number Pi with a different algorithm based on the so called Monte Carlo method. Monte Carlo methods are a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results. The idea is to simulate random (x, y) points in a 2-D plane, being the domain a square of side 1 unit. Imagine a circle inside the same domain with same diameter and inscribed into the square. We then calculate the ratio between the number of points that lied inside the circle and the total number of generated points. The higher the number of samples, the better the precision obtained. Note that the cost and execution time of this algorithm and the one used earlier on in this assignment are not directly comparable since they use different methods for computing number Pi. We use this third algorithm in this section for two reasons: 1) to show an example of a Monte Carlo method, a class of widely used algorithms which are costly but highly parallelizable; 2) to have a code which takes longer to execute and has enough work to scale reasonably when using MPI processes. Note that managing processes has higher costs and overheads than dealing with threads.

1.5.1 Strong scalability with processes

Submit a job to the execution queue with `sbatch submit-strong-mpi.sh` in order to get a plot of the execution time and speed-up that is obtained when varying the number of MPI processes (strong scalability). **Did the execution scale well up to 32 MPI processes? Inspect file `speedup.txt`.** Which is

⁸In the boada Linux distribution you can use the ghostscript `gs` command to visualise Postscript files or convert the files to PDF using the `ps2pdf` command and use `xpdf` to visualise PDF files.

the speedup with 1 MPI process. A number lower than 1 indicates a slower execution. If you inspect file `submit-strong-mpi.sh` you'll see that we are comparing the execution time of the MPI program run with a different number of processes against the original sequential code. The latter does not incur in the overheads of using MPI and runs faster than the MPI code run with a single process. Compare now the speedup, or slowdown, obtained with 1 MPI process to the one obtained earlier with a single OpenMP thread. The sequential execution of a parallel program incurs overheads. Which of MPI and OpenMP introduces more overheads? You can open the output file `submit-strong-mpi.sh.oJOB_NUMBER` and observe the execution time of the original sequential code `pi_seq` and the MPI code `pi_mpi_collectives` using a single MPI process. Is the latter much slower? Similarly, in the OpenMP directory you observe the difference between the execution of the original sequential code `pi_seq` and the OpenMP code `pi_omp` using a single thread at the beginning of the output file `submit-strong-omp.sh.oJOB_NUMBER`. Is the latter much slower? Note that we focus here in the parallelization overheads of MPI and OpenMP. However, we are not comparing the very different execution times of two sequential implementations which implement a different method for computing Pi.

1.5.2 Strong scalability with processes and threads

In addition, we want to observe the combination of MPI and OpenMP. Execute `sbatch submit-mpi2-omp.sh` and study the results presented in the output file created after the execution finishes. Note that since the test uses 2 MPI processes, each of them using 1 or more threads, the execution implies using several even number of threads between 2 and 64. There are between 1 to 32 threads per MPI process; with 2 processes this results in having twice as many threads.

Was the execution with 32 MPI processes faster or slower than the one with 2 MPI processes each using 16 OpenMP threads? Which is the speedup of the parallel code run with 2 MPI processes each using 32 OpenMP threads with respect to the sequential Monte Carlo code without MPI?

Does this match the results observed at the end of section 1.4.3 when we changed the value for `np_NMAX` in the `submit-strong-omp.sh` from 32 to 64? Did the execution improve in that case? According to the execution of `sbatch submit-mpi2-omp.sh`, does it improve in this case? Can you explain the different behaviour? *Hint:* you may edit `submit-mpi2-omp.sh` and change the configuration of Slurm to use a single node (`#SBATCH --nodes=1`) and run again. Compare the new results with the previous ones. Please, do not run experiments with more than 2 nodes since no other jobs could run in the execution partition in the meanwhile and you would collapse the system.

Managing MPI processes has larger overheads than managing OpenMP threads. However, MPI allows us to use more memory in a distributed memory system. We hope that, with this experiments, you realize that we can speed up the code by parallelizing a code with OpenMP, with MPI, or a combination of both to use several nodes and several cores per node. Also, that to achieve high performance it's important to understand the architectures, the algorithms, and how to use the programming models.

Deliverable

After completing this laboratory assignment, and before starting the next one, you will have to deliver a report in PDF format (other formats will not be accepted) containing the answers to the questions stated at the end of this document;

Your professor will open the assignment at the moodle Atenea website and set the appropriate delivery dates for the delivery. Note that, unlike the subsequent assignments which can be done in groups of 2 students, **this initial assignment must be done independently by each student, requiring Individual submission** through the moodle Atenea website.

VERY IMPORTANT:

For this assignment and all the subsequent ones:

1. Once the deadline is reached Atenea will close automatically and **late submissions by email will not be accepted.**
2. When asked to submit source files, they must be packed with zip. The zip file must contain only the source codes. You must submit your report as a pdf file. The pdf file with your report must be uploaded to Atenea separately, not within the zip file. **Failure to do so will subtract 3 points out of 10.**
3. In the front cover of the document, please clearly state your name and surname, the identifier of the group (username `cpds1XYZ`), title of the assignment, date, academic course/semester, ... and any other information you consider necessary. **Failure to do so will subtract 3 points out of 10.**
4. As part of the document, you can include any code fragment, figure or plot you need to support your explanations. Many plots generated with the scripts contain a caption with the username and time of creation which must be kept with the plots when added to your report. **Failure to do so will imply no points obtained for that question.**
5. When you are asked to explain or justify some answer to a question, you must do it briefly with your own words and using the information available to you. **Failure to explain properly will imply no points obtained for that question.**
6. Laboratory assignments are not an exam. While you are working on the assignments feel free to ask questions, always after reading properly and doing your part of the work. **Solve your doubts in time so that you get ready for the laboratory exam.**

In case you need to transfer files from boada to your local machine (laptop or desktop in laboratory room), or viceversa, you can use the secure copy `scp` command. For example "`scp -O cpds1XYZ@boada.ac.upc.edu:a0/foobar.txt local/directory/.`" executed in your local machine to copy file `foo.txt` inside directory `a0` in your home directory of boada to directory `local/directory/` with the same name.

Node architecture and memory

1. Complete the following table with the relevant architectural characteristics of the node types available in `boada-[18-20]`:

Characteristic	Value
Number of 'NUMA nodes' per node	
Number of sockets per node	
Number of cores per socket	
Number of threads per core	
Maximum core frequency	
L1-I cache size (per core)	
L1-D cache size (per core)	
L2 cache size (per core)	
Last-level cache size (per socket)	
Main memory size (per socket)	
Main memory size (total within node)	

2. Include in the document the architectural diagram for one of the nodes `boada-18` to `boada-20`.
3. Each of the nodes `boada-*` is a **NUMA node**. What does it mean? Which are the normalized distances, or latencies, between them? Based on the ACPI System Locality Information Table (SLIT) which shows the relative cost of local and remote memory accesses, state the ratio of local to remote memory access latency in simplified form (1:X) so that we easily understand how many times slower an access to the memory of another NUMA node is w.r.t. accessing the memory in the local NUMA node.

Timing sequential and parallel executions

For each of the following sections show the plots or tables required. But, in addition, please **provide some reasoning about the results obtained**. For instance, comment whether the scalability is good or not, and why.

3. Plot the execution time and speed-up that is obtained when varying the number of threads (strong scalability) by submitting the jobs to the `execution` queue (section 1.4.3). Show the parallel efficiency obtained when running the weak scaling test. Explain what strong and weak scalability refer to, exemplifying your explanation with the plots that you present. Explain the reasons which cause the behaviour observed when more than 32 threads are used.
4. Plot the execution time and speed-up that is obtained when varying the number of **MPI** processes from 1 to 32 (strong scalability) by submitting the jobs to the `execution` queue (section 1.5.1). In addition, show in a table the elapsed execution time when executed with 2 MPI processes when varying the number of threads from 1 to 32 (section 1.5.2) ⁹. Which was the original sequential time?; the time with 32 MPI processes?; and the time with 2 MPI processes each using 32 `OpenMP` threads? And the execution time for this latter case when using a single node of `boada` instead of using 2? You can retrieve such information for 1 and 32 MPI processes from file `elapsed.txt`; and for 2 MPI processes each with 32 threads within the output file created after the execution of `sbatch submit-mpi2-omp.sh`.

⁹Note that for 2 MPI processes the scalability test implies using an even number of threads between 2 and 64.